

MICROPROCESOARE SI LIMBAJE DE ASAMBLARE

Calculatoare 3 2025-2026

Curs 5 Sapt 5 28 octombrie 2025 16:00-18:00

serban@upit.ro

Modul de notare:

Examen:	50%	
Laborator:	30%	A2
Test (Midterm):	20%	A1

Planificarea activitatilor:

Laborator:

- saptamanal, conform orarului;
- sapt. 13, 14 refaceri laboratoare;

Curs:

- saptamanal, conform orarului;

Test (Midterm): in saptamana a 10-a (2 decembrie - 6 decembrie)

OBS. Este nevoie de nota minima 5 la toate activitatile (semestru – A1, A2 si sesiune – Examen!!! pentru promovare)

Continut

Introducere

Evolutia microprocesoarelor

Criterii de clasificare ale microprocesoarelor

Structura microprocesoarelor

Mecanisme specifice Microprocesoarelor

Microprocesoare pe 8 biti

Familia de microprocesoare x86

Familia de procesoare ARM

Bibliografie

The Design of a Microprocessor – Wilhelm G. Spruth – Springer

Microprocessor Design – Grant McFarland – McGraw-Hill – 2006

Microprocessor Interface Design – J.D. Nicoud – Chapman & Hall

Microprocessors and Microcomputers Hardware and Software - Ronald J. Tocci – Pearson

Intel Microprocessors – Barry Brey

Z-80 Microprocessor Architecture, Interfacing, Programming and Design - Ramesh Gaonkar

Introduction to 80X86 Assembly Language and Computer Architecture - Richard C. Detmer - Jones & Bartlett Pub (2001)

X86 Assembly Language and C Fundamentals – Joseph Cavanagh – CRC Press

ARM Assembly Language Fundamentals and Techniques, Second Edition - William Hohl

Fundamentals of Embedded Software Where C and Assembly Meet – Daniel Wesley Lewis

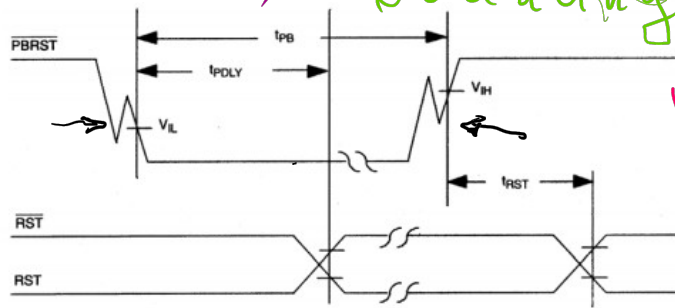
Mecanismul de RESET

- Reset manual
- Power On Reset POR
- Supervisor
- Brown-out
- Watchdog

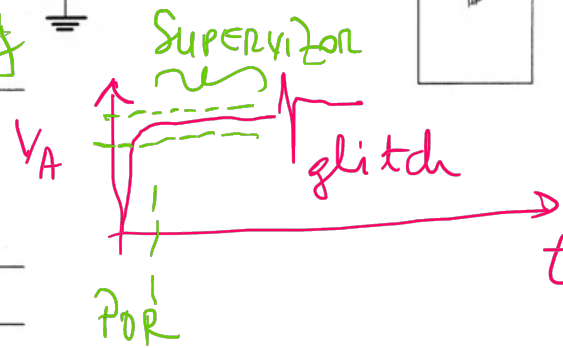
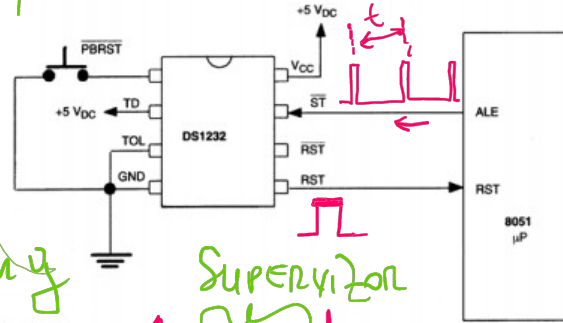
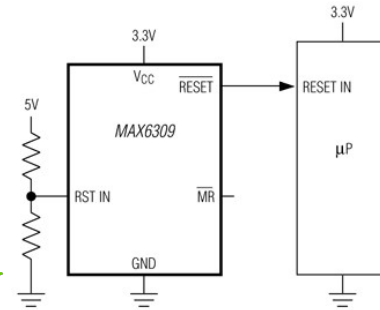
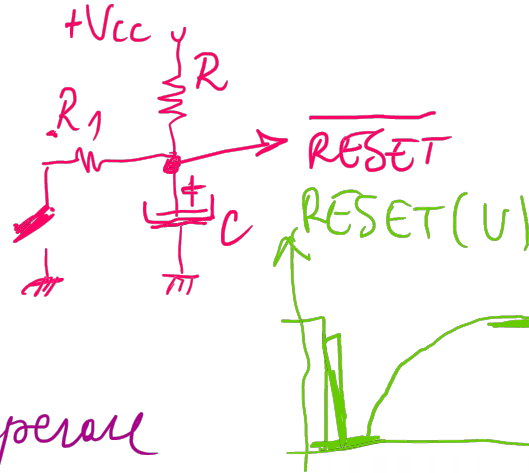
Condiție inițială de operare
(prin reset)

$PC \leftarrow \text{adr. start (reset)}$

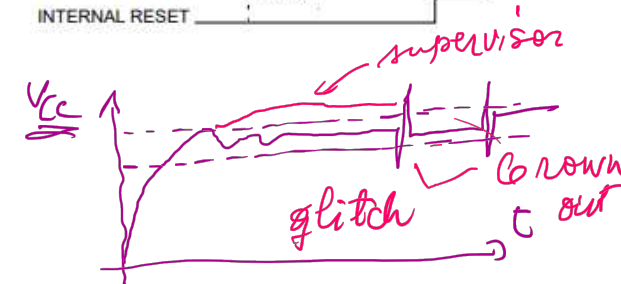
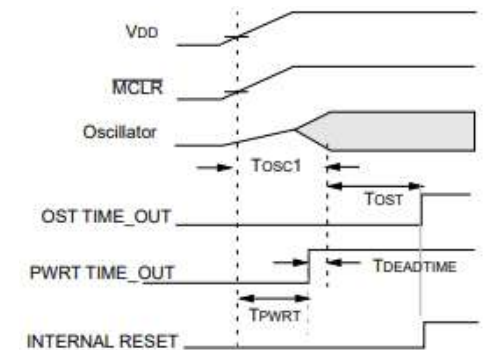
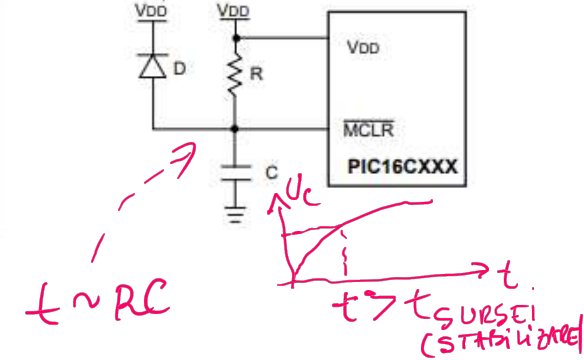
(μP z-80 $\rightarrow$ 0000h)
(μP x86 $\rightarrow$ FFFFFFFFh)
32bit

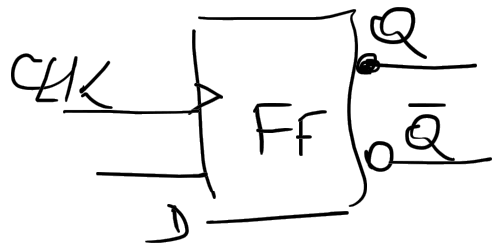


Mecanisme specifice in procesoare

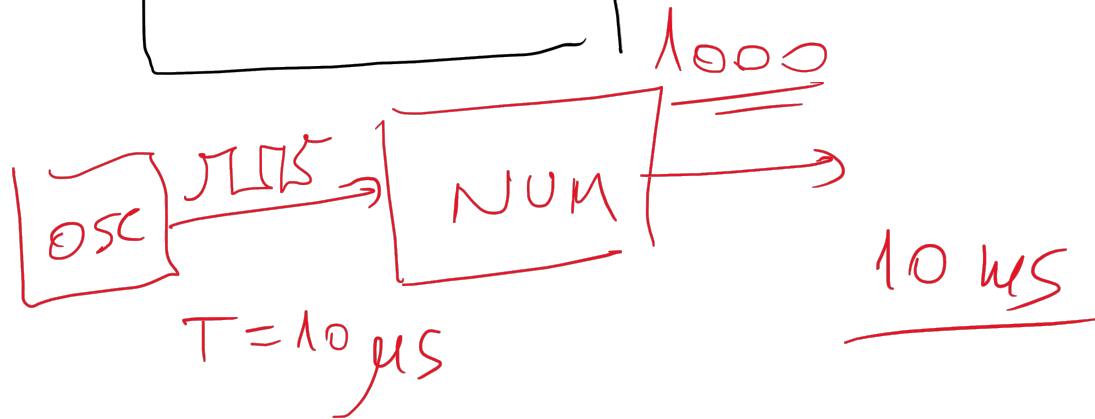
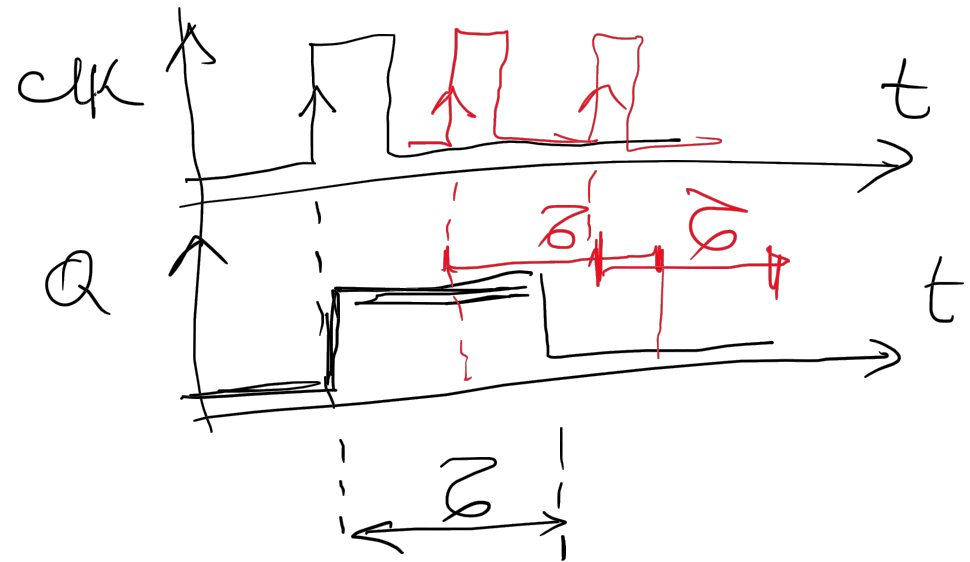
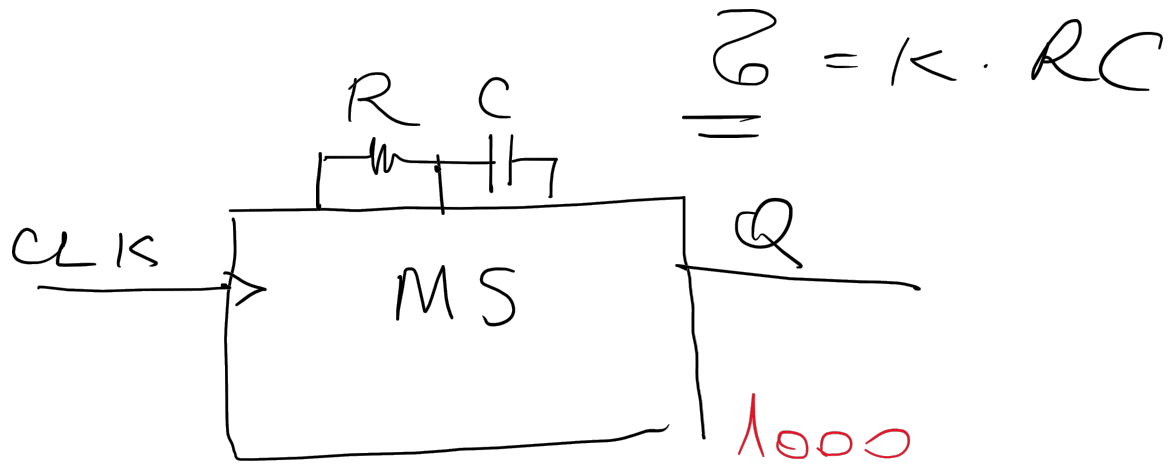


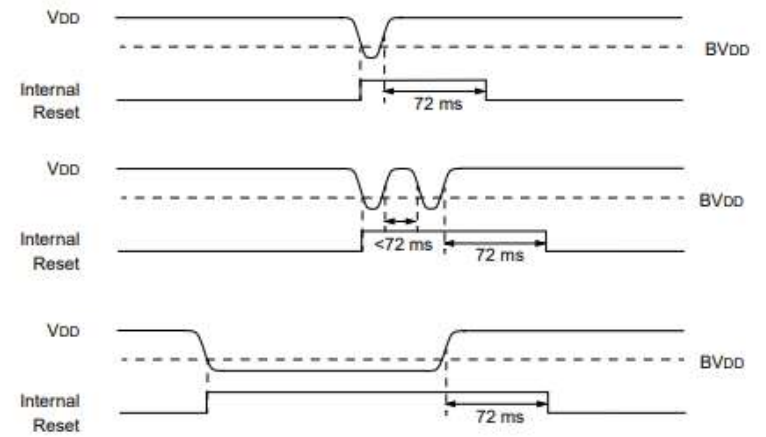
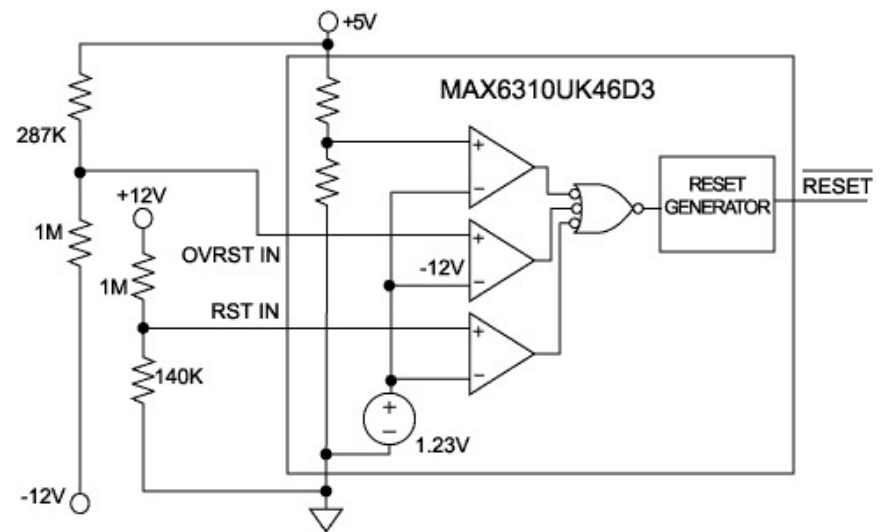
$$U_{nom} \in [0,5 \dots 0,9] U_{abs} \quad U_{nom} \approx 5V \quad 10\mu F / 25V \quad 10\mu F / 10V \leftarrow U_{abs}$$

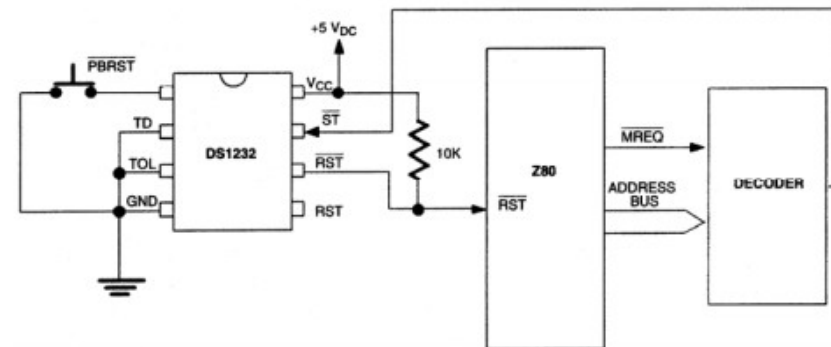
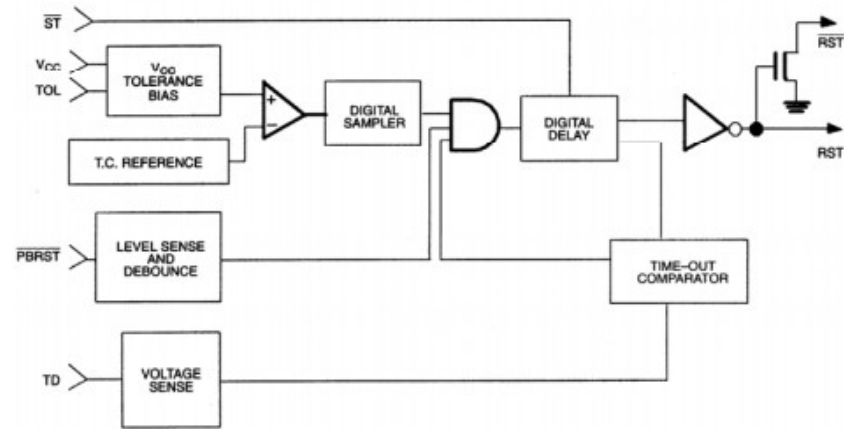
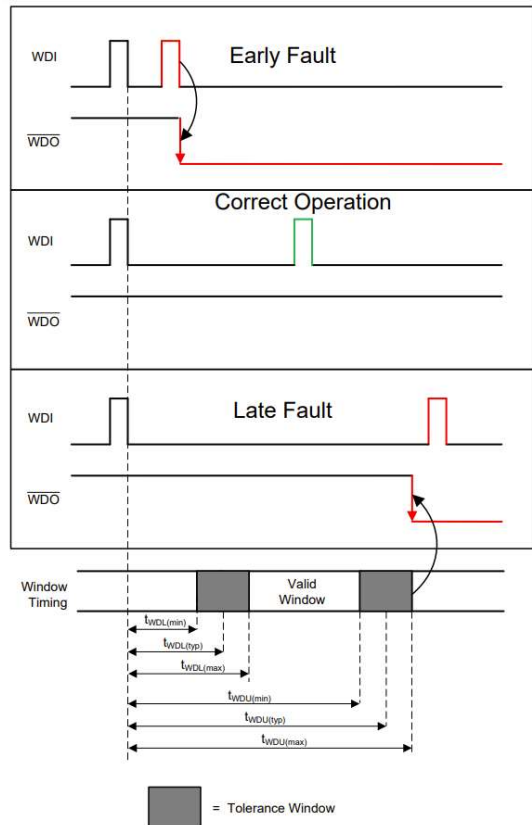




flip-flop







Mecanismul de operare cu stiva si registrul SP

Stiva este o zonă de memorie de tip RAM, de obicei externă procesorului, folosită pentru:

- situația apelării subrutinelor; o subrutina este formata dintr-un grup de instructiuni a caror rulare se repeta intr-un program; pentru a nu ocupa memorie, grupul respectiv de instructiuni se individualizeaza intr-o zona de memorie, marcandu-se adresa de inceput printr-o eticheta, iar sfarsitul logic printr-o instructiune specializata de revenire (RETURN); subrutina va fi apelata din program de fiecare data cand este necesar printr-o instructiune de tip CALL eticheta (adresa subrutinei).
- tratarea întreruperilor și excepțiilor programului de lucru. Întreruperile respectiv excepțiile sunt evenimente deosebite care apar în timpul execuției. O întrerupere este o semnalizare făcută procesorului de dispozitive externe acestuia (circuite de I/O). Spre exemplu, scurgerea unei perioade de timp poate fi semnalizată printr-o cerere de întrerupere. Excepțiile sunt semnalizări adresate procesorului de anumite componente interne, de exemplu forțarea operației de împărțire prin 0. Pentru toate situațiile specificate proiectantul software-ului trebuie să scrie subrutine pentru tratarea situațiilor deosebite apărute. Trecerea la tratarea situațiilor menționate se face de către procesor prin apeluri speciale ale subrutinelor scrise în acest scop.

OBS. Situațiile mentionate mai sus folosesc stiva în mod automat.

- Salvarea / memorarea de informații la cererea programatorului sau automat (continuturi de registri, de locații de memorie) prin executia de instructiuni specifice (PUSH la depunerea în stiva, POP la extragerea din stiva).
- Schimburi de continuturi între registri.

Stiva si registrul SP

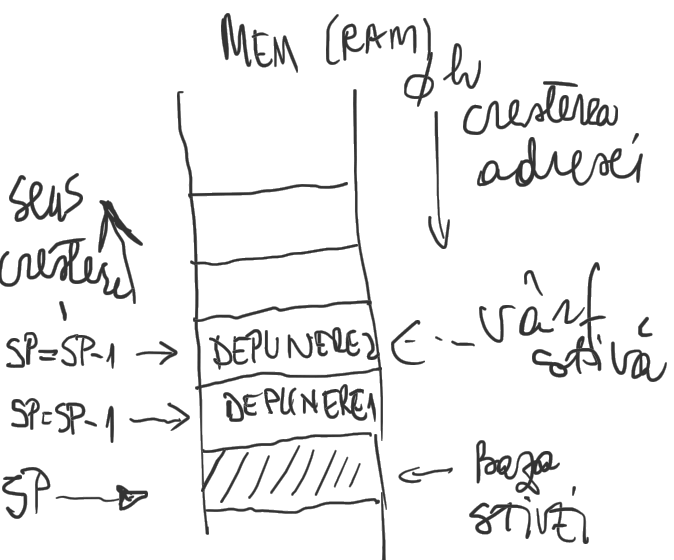
Stiva este adresată printr-o tehnică LIFO (last in – first out) prin utilizarea registrului SP (stack pointer – indicatorul varfului stivei). În practică se întâlnesc două modalități de creștere a stivei (la depunerea în ea):

- ✓ Către adrese mici de memorie/înapoi; În această situație la fiecare depunere în stivă, SP se decrementează (la extragere SP se incrementează);
- ✓ Către adrese mari; În acest caz la depunere SP se incrementează, iar la extragere SP se va decrementa.

SP-ul arată întotdeauna varful stivei, adica adresa ultimei depuneri (sau a bazei stivei). Există și posibilitatea fixării bazei stivei. Aceasta se face prin încărcarea SP-ului cu adresa absolută de la care urmează să crească stiva.

Stiva poate fi internă sau externă procesorului cu care operează, astfel cele mai multe procesoare pot forma stivă în memoria externă adresabilă. Dat fiind faptul că stiva trebuie scrisă, respectiv citită, zona de memorie în care acesta este implementată este de tip RAM. Pentru această situație (stivă externă) dimensiunea stivei depinde de limitele memoriei și de spațiile alocate în aceasta pentru diversele aplicații.

Există și procesoare la care stiva se formează în zone de memorie special alocate și aflate pe același cip. Dimensiunea acestor zone de memorie specifice este limitată, doar câteva locații disponibile (de ordinul zecilor). Procesoarele aflate în această situație sunt astfel limitate în posibilitățile lor numărul de depuneri ce se pot face în stivă fiind mărginit de numărul de locații al acesteia.



SP - se decrementează
la depunerea în stivă

SP - se incrementează
la extragerea din stivă

Utilizatorul “simte” acest lucru prin limitarea numărului de subrutine imbricate (apelate unele din altele) pe care le poate folosi. În această categorie intră procesoarele realizate de firma MICROCHIP, denumite PIC.

Nu există procesoare care să dispună de ambele modalități de formare a stivei, adică atât în memoria externă cât și în cele interne dedicate. Pentru ambele categorii de procesoare relativ la modul de formare a stivei accesarea prin utilizarea registrului SP se poate face fie prin decrementarea SP-ului, fie prin incrementarea acestuia (la depunerea în stivă). Pentru un anumit procesor este bine definit acest mecanism de operare cu stiva utilizând registrul SP.

De obicei, salvarea în stivă a conținuturilor registrilor se face la începutul derulării subrutinelor, scopul fiind acela de a memora informațiile din ei, în ideea eliberării acestora pentru executia subrutinei. Salvarea se face manual prin instrucțiuni PUSH ale utilizatorului. La sfârșitul subrutinei, se restaurează din stiva conținuturile anterioare ale registrilor, în ordine inversă depunerii, folosind instrucțiuni de tip POP. Se respectă tehnica de adresare LIFO a stivei.

Exemplu de salvare si restaurare in si din stiva folosind instructiunile PUSH si POP

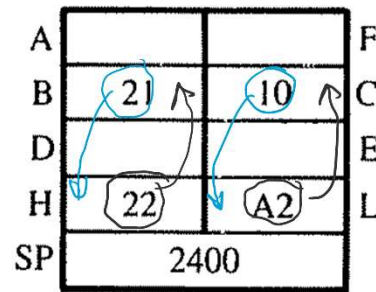
*

PROGRAM

2000 LD SP, 2400H (Baza stivei) LD SP, 2400H
 2003 LD HL, 22A2H
 2006 LD BC, 2110H
 2009 LD A, (HL)
 200A OR A
 200B ----->

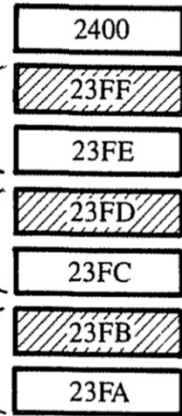
2010 PUSH BC
 2011 PUSH HL
 2012 PUSH AF

2035 POP AF
 2036 POP HL
 2037 POP BC



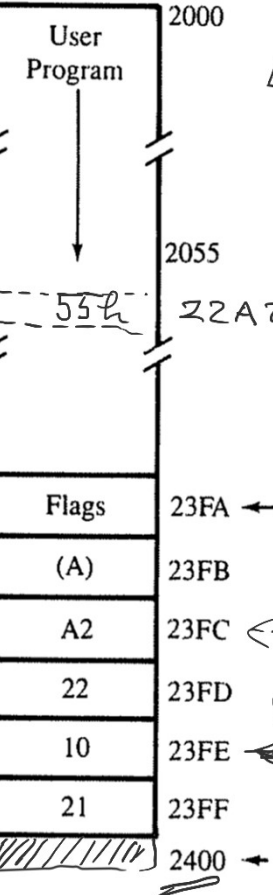
LITTLE ENDIAN

Stack Pointer Contents



OR A - $A \leftarrow A \vee A$

Memory Contents



LD A, (HL)
 $A \leftarrow 55h$

LIFO

28 Oct 2025